

Global Illumination

(Indirect Lighting)



$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} f_r(\mathbf{x}, \omega_i, \omega_o) L_i(\mathbf{x}, \omega_i) (\omega_i \cdot \mathbf{n}) d\omega_i$$

[보이는 최종 색]

[스스로 내는 빛]

[들어온 빛이 재질/각도에 맞게 튕겨나온 양]

“ 보이는 색 = 자기 발광 + 주변 빛의 반사 결과 ”

표면의 최종 색은 자기 발광 성분과, 모든 입사 방향의 빛이 재질과 입사각에 따라 반사된 결과의 합으로 결정됩니다.



Rendering Equation

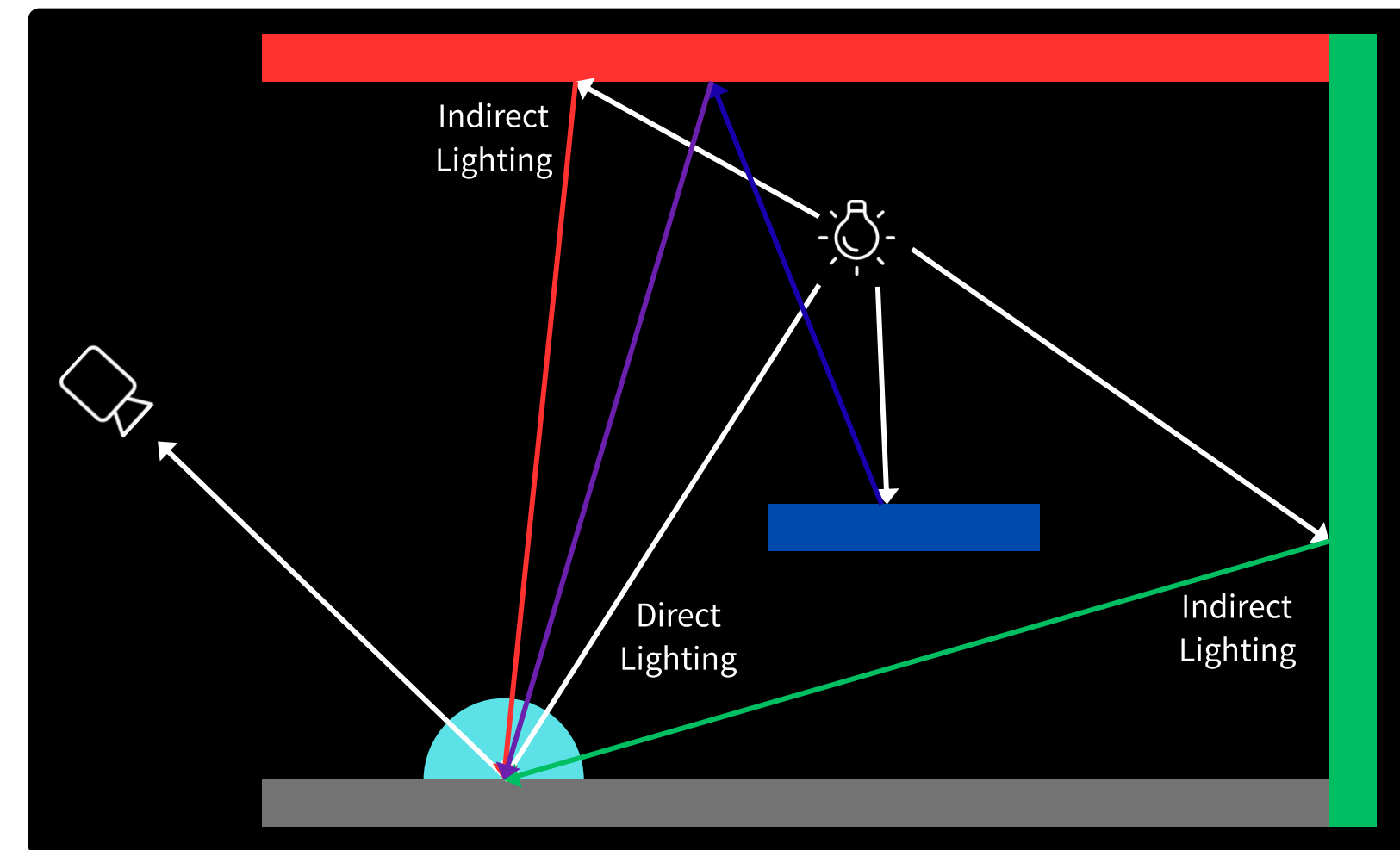
origina

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} f_r(\mathbf{x}, \omega_i, \omega_o) L_i(\mathbf{x}, \omega_i) (\omega_i \cdot \mathbf{n}) d\omega_i$$

simplified (Phong Lighting)

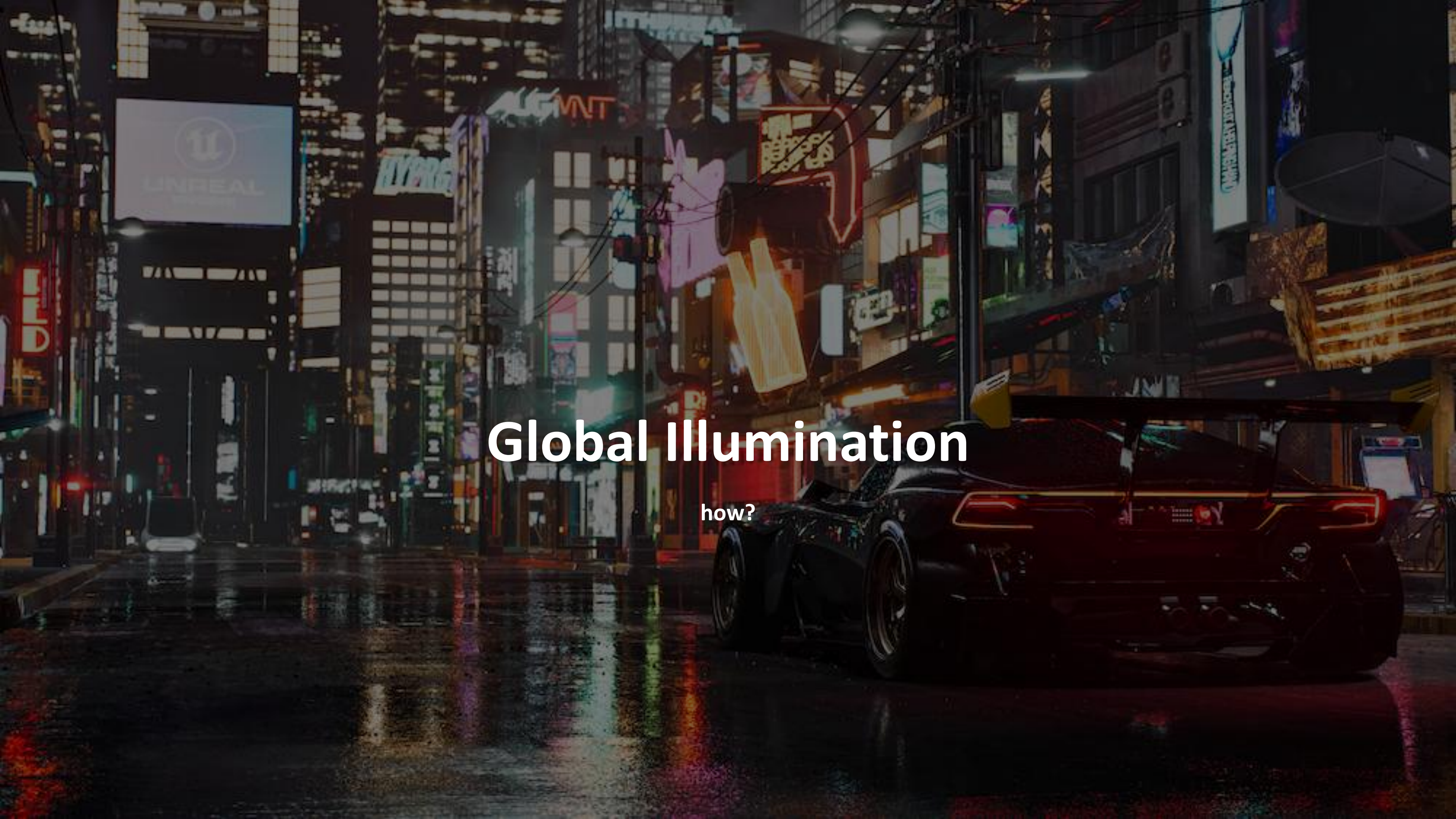
$$color_{\{R,G,B\}} = \underbrace{m_{\{R,G,B\}}^{ambient} \cdot l_{\{R,G,B\}}^{ambient}}_{\text{ambient}} + \underbrace{\sum_{i=1}^{num_lights}}_{\text{attenuation}} \underbrace{\frac{1}{k_c + k_l d_i + k_q d_i^2}}_{\text{attenuation}} \underbrace{\left(m_{\{R,G,B\}}^{diffuse} \cdot l_{i,\{R,G,B\}}^{diffuse} \cdot \max(L_i \cdot N, 0) \right)}_{\text{diffuse}} + \underbrace{m_{\{R,G,B\}}^{specular} \cdot l_{i,\{R,G,B\}}^{specular} \cdot \max(R_i \cdot V, 0)^{shininess}}_{\text{specular}}$$

Considers lights only!



Global Illumination

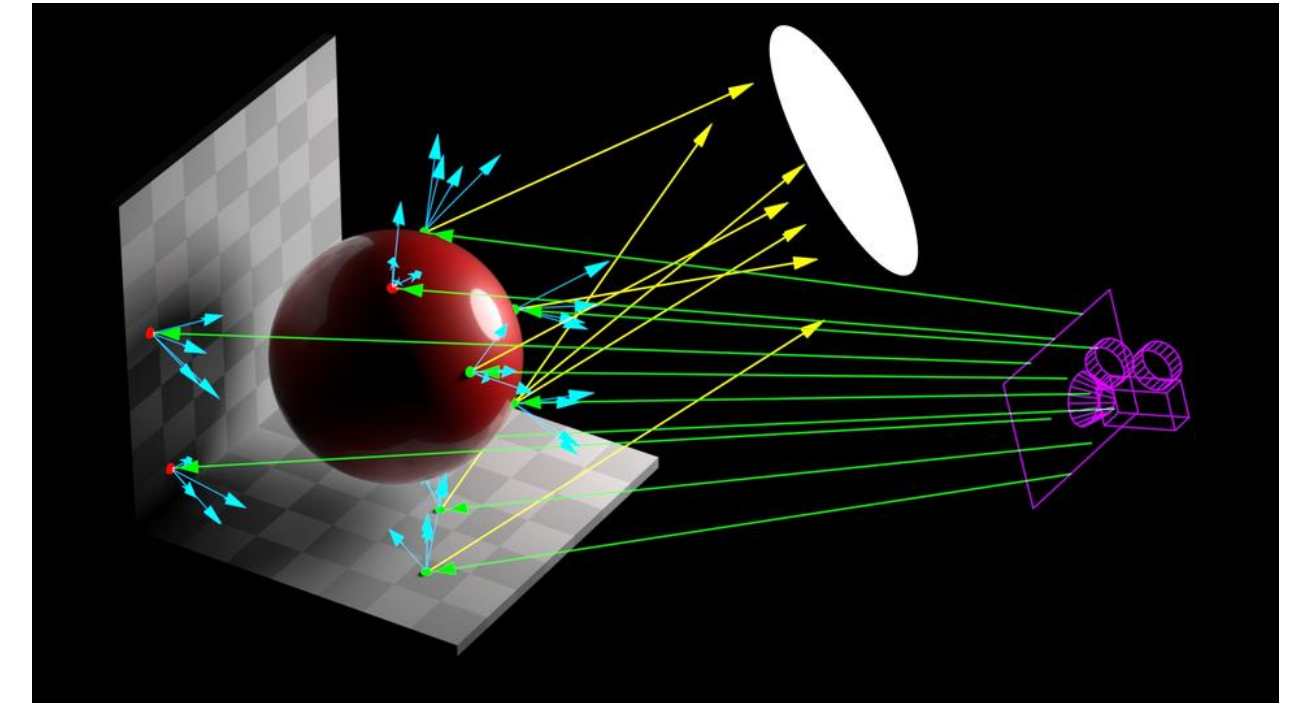
how?



Offline Global Illumination

Path Tracing

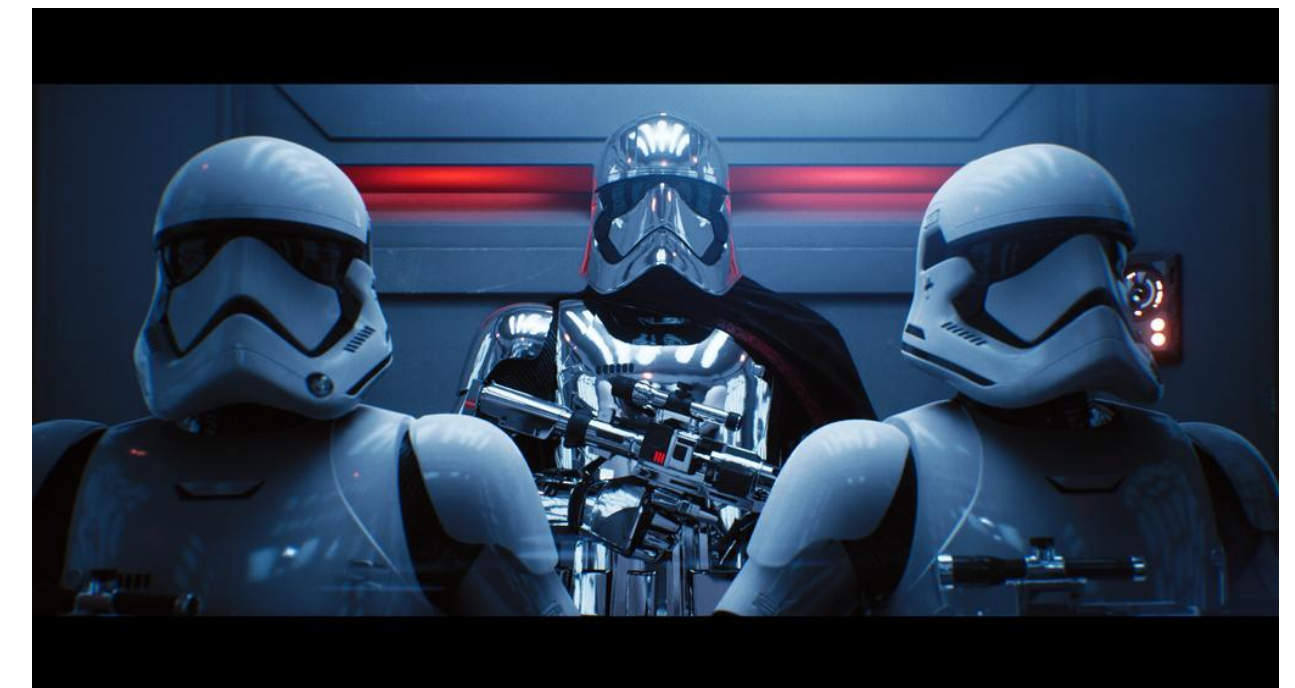
- Shoot **multiple rays** from the camera through **each pixel**
- Intersect rays with scene geometry
- **Bounce** rays across surfaces **multiple times**
- Sample incoming light at each bounce
- Accumulate direct and indirect lighting
- Average the results of many ray paths
- Approximate global illumination through light transport simulation



Total Ray Count=(Screen Resolution) × (Rays Per Pixel) × (Bounce Count)

$$1920 \times 1080 \times 512 \times 8 \approx 8.5 \text{ billion rays}$$

High computational cost, but delivers highly accurate and realistic lighting
The industry standard for **film** and **CGI rendering** (Pixar, Disney, etc.)
(often requiring minutes to hours per frame on render farms)



But... in real time?

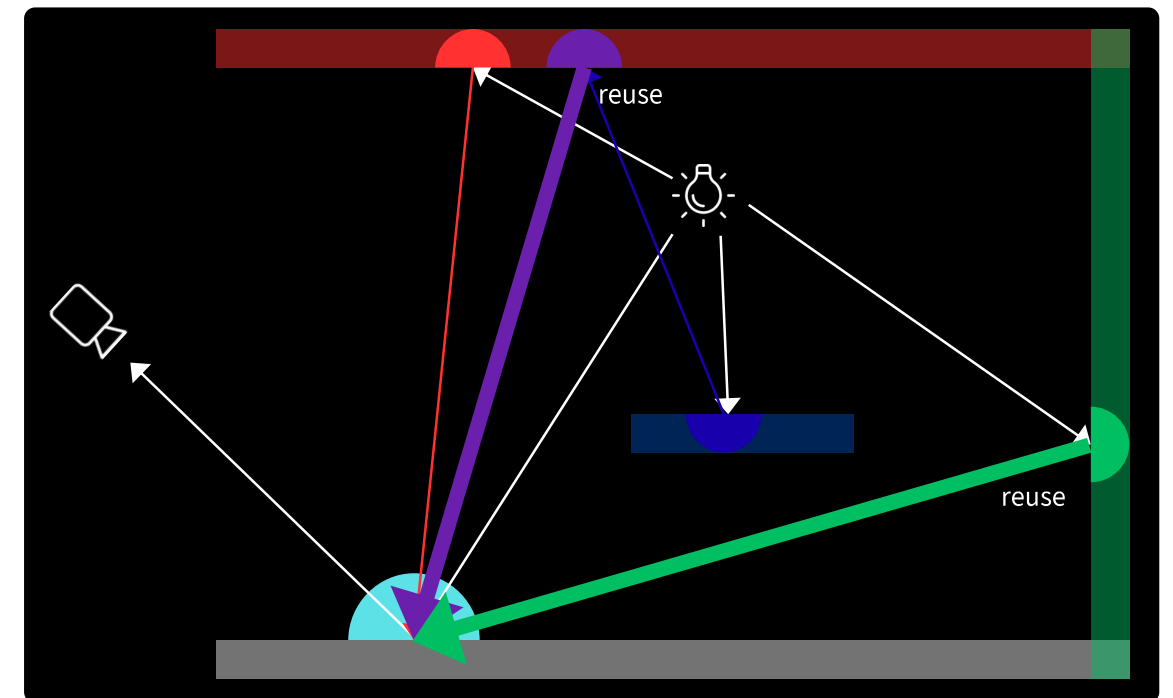
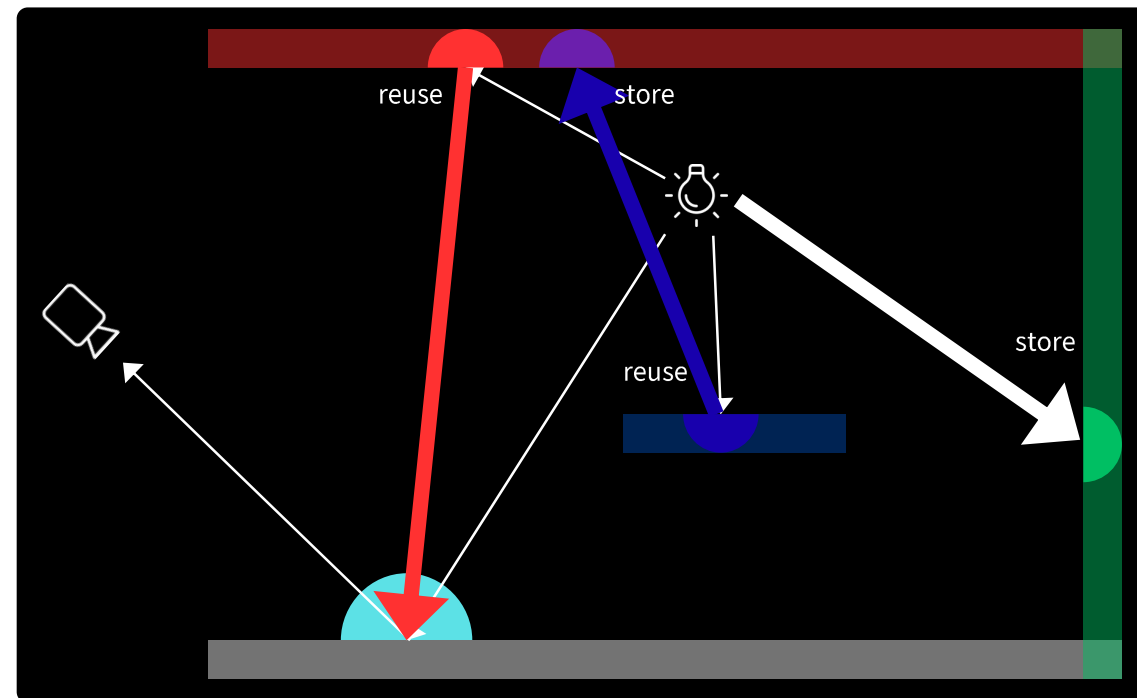
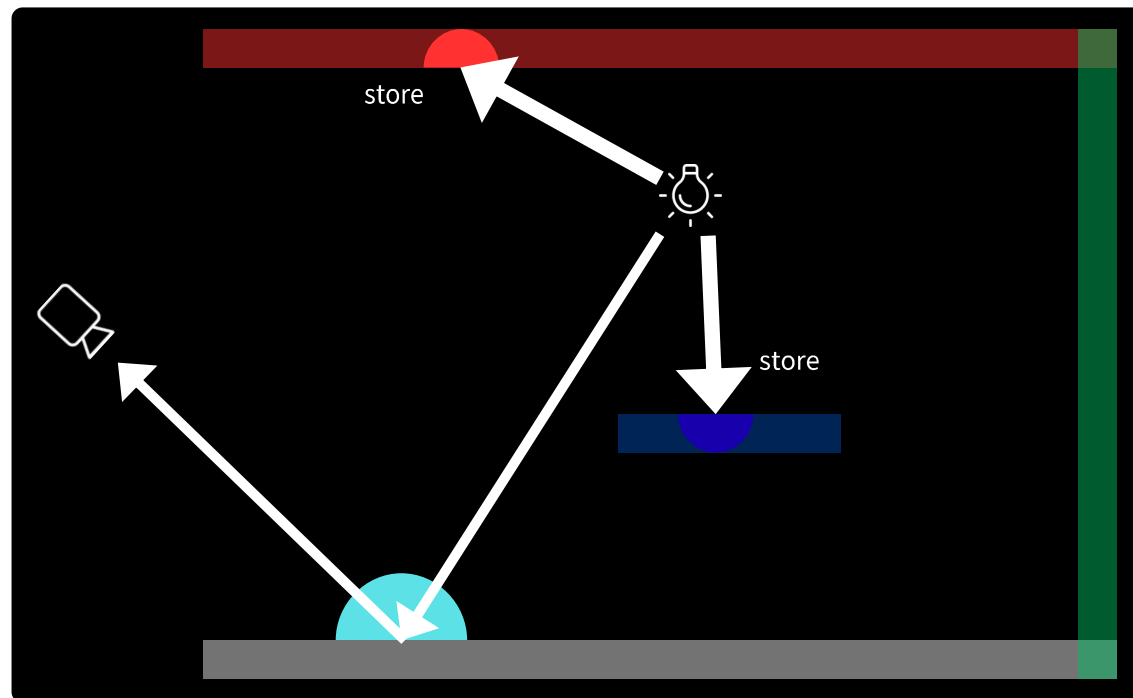
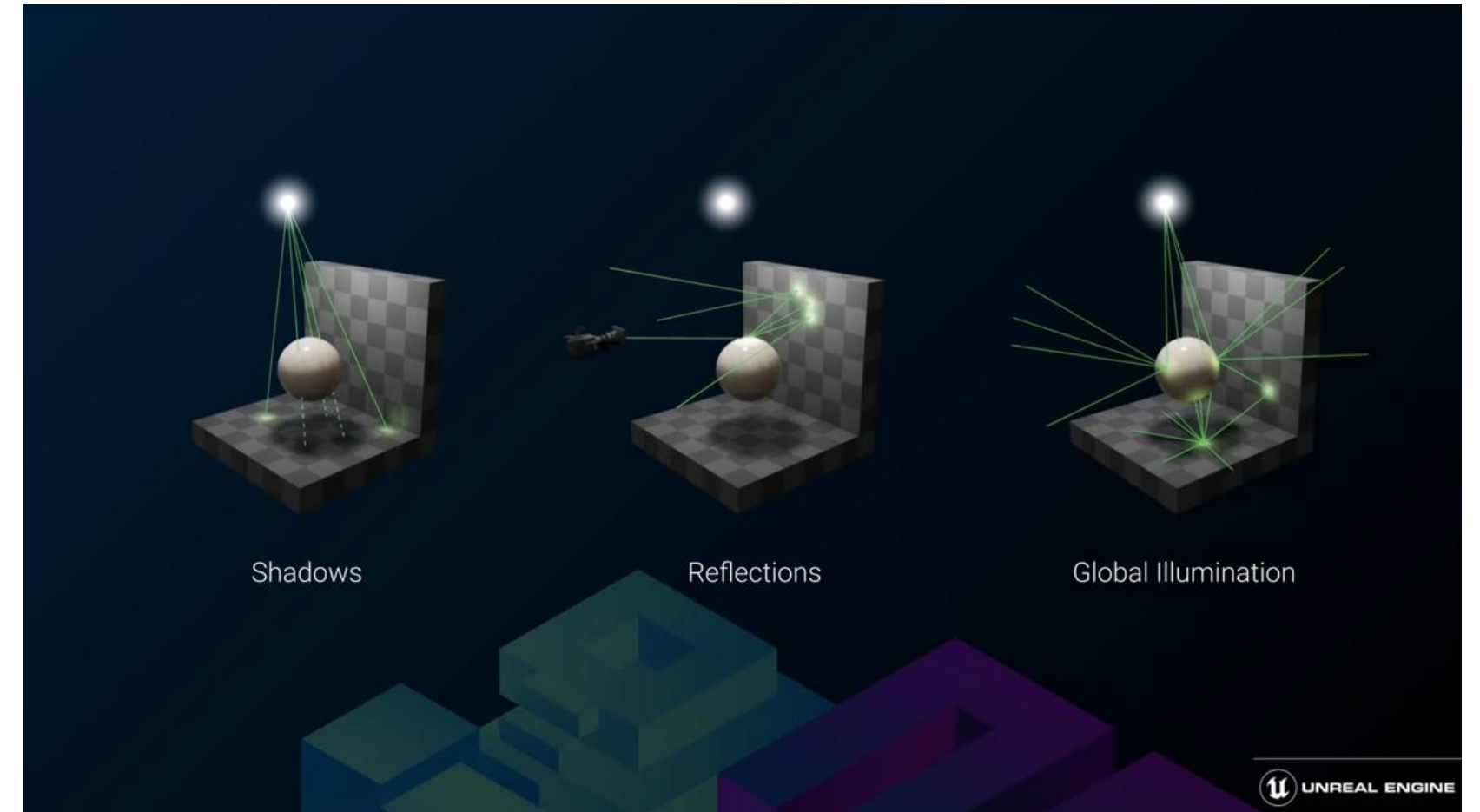
Real-time Global Illumination

Reuse and Cache Lighting Information

Instead of computing all light bounces every frame, light is progressively propagated and accumulated across **multiple frames**.

Different GI techniques are mainly distinguished by how this lighting information is **stored** and **reused**:

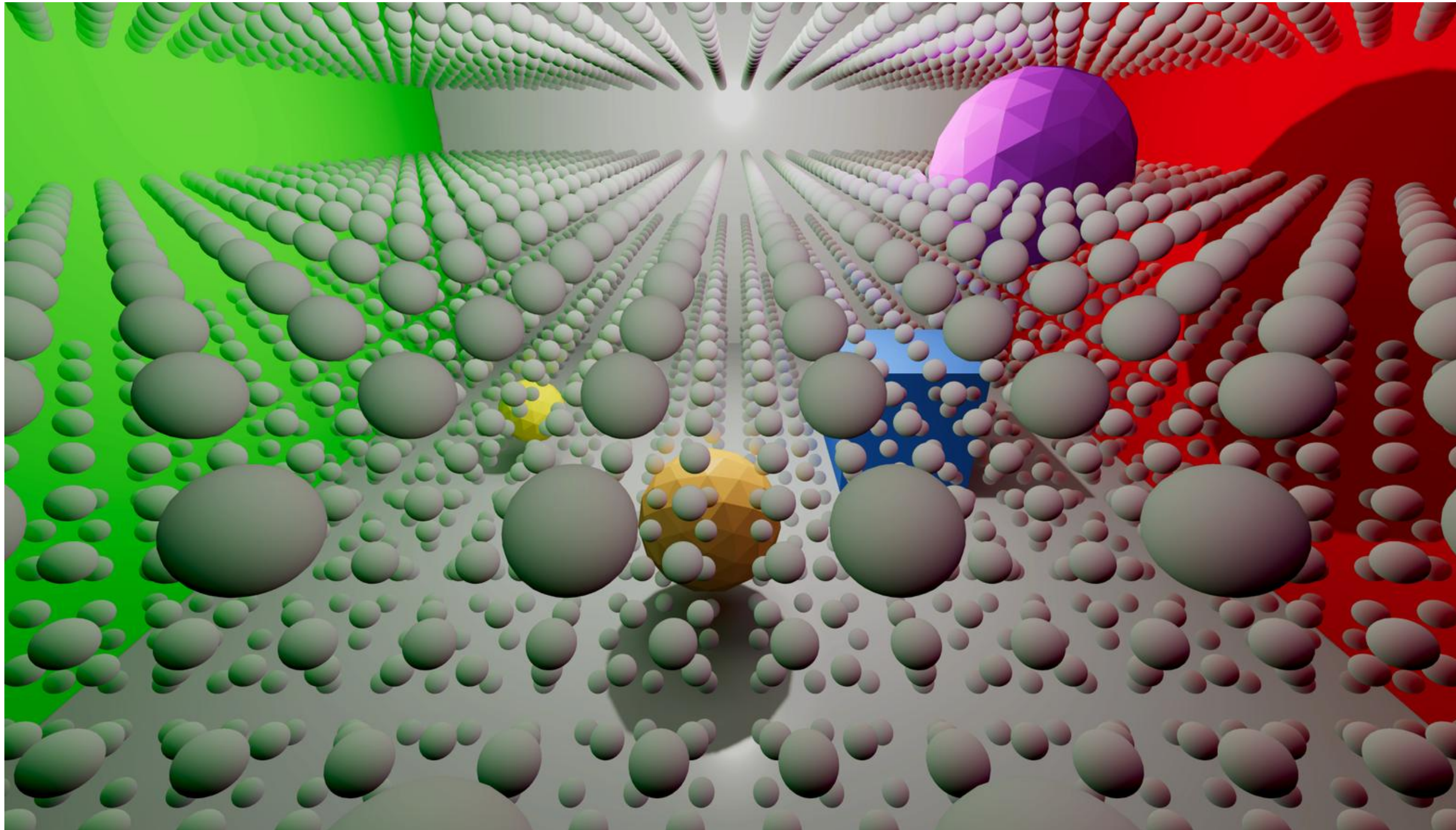
- **Probe-based** GI (DDGI)
- **Voxel-based** GI (Voxel GI)
- **Surfel-based** GI (Surfel GI)



...

Dynamic Diffuse Global Illumination

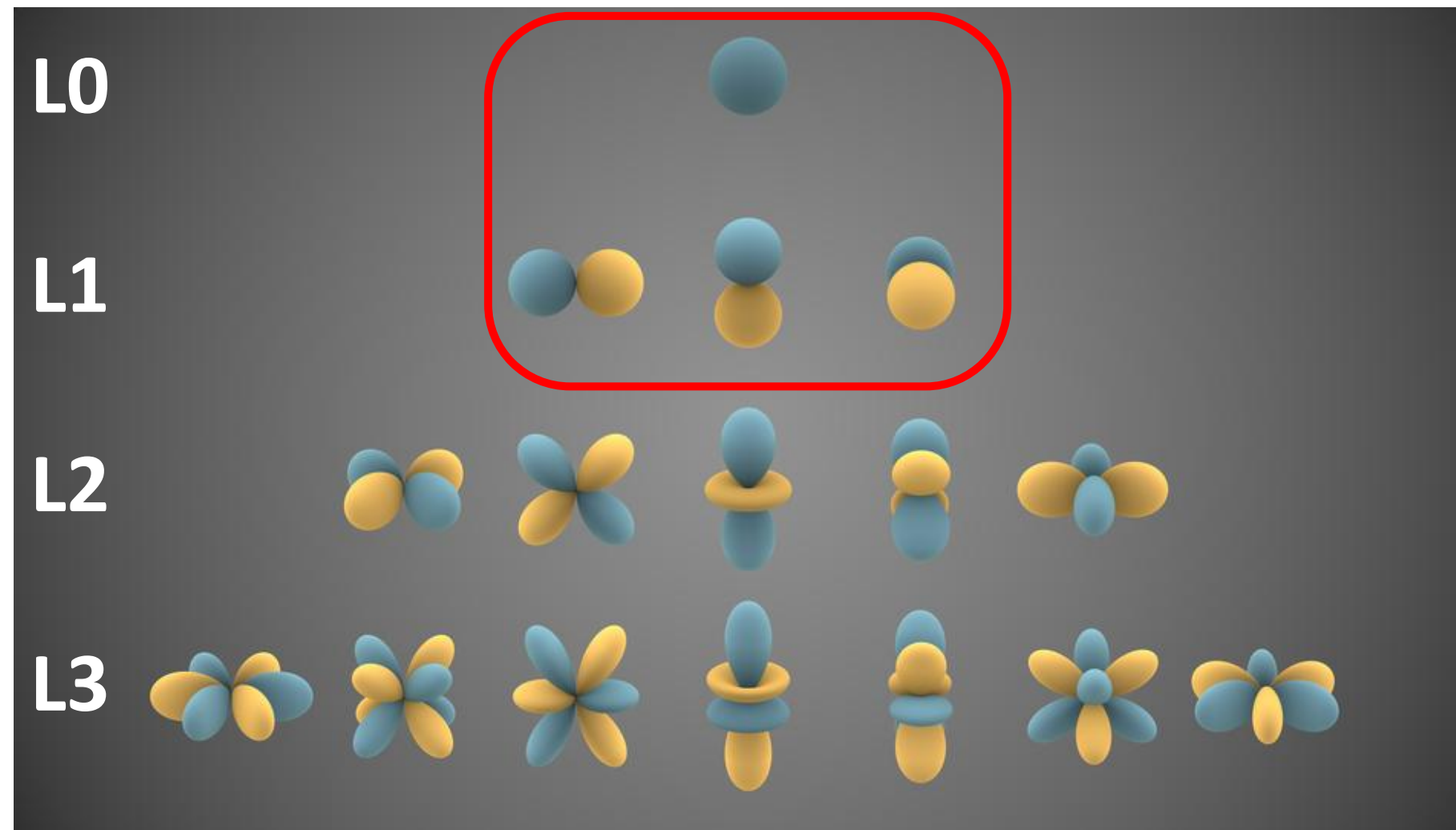
Dynamic Diffuse Global Illumination (DDGI)



1. Ray tracing from each probes → Capture radiance
2. Store as **Spherical harmonics(SH)** per probe
3. Reuse at Ray tracing from each probe again! → **Recursive** feedback
4. Infinite bounces approximated across frames

Dynamic Diffuse Global Illumination

Spherical harmonics



Spherical Harmonics (SH) represents a directional signal as a weighted sum of simple basis functions.

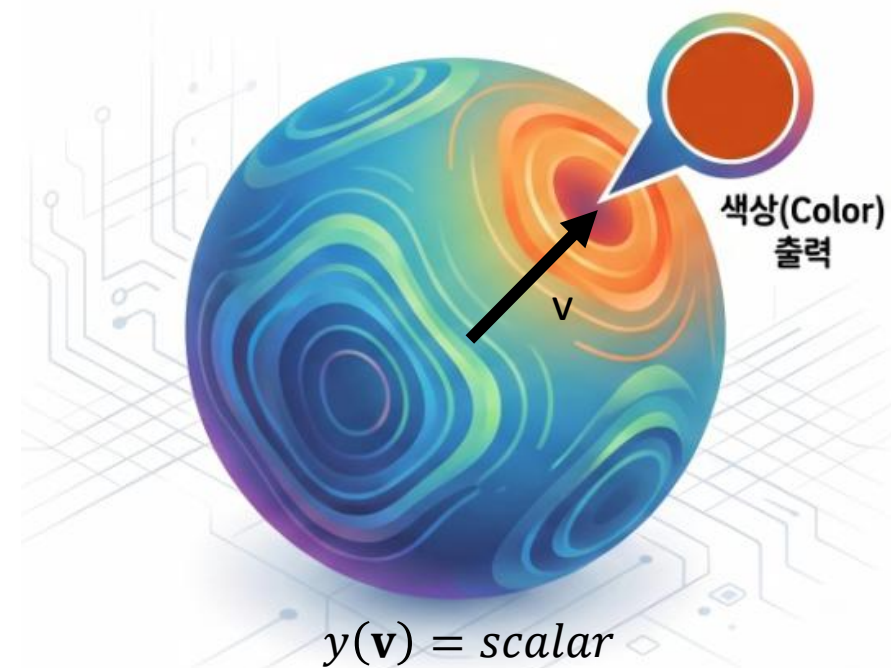
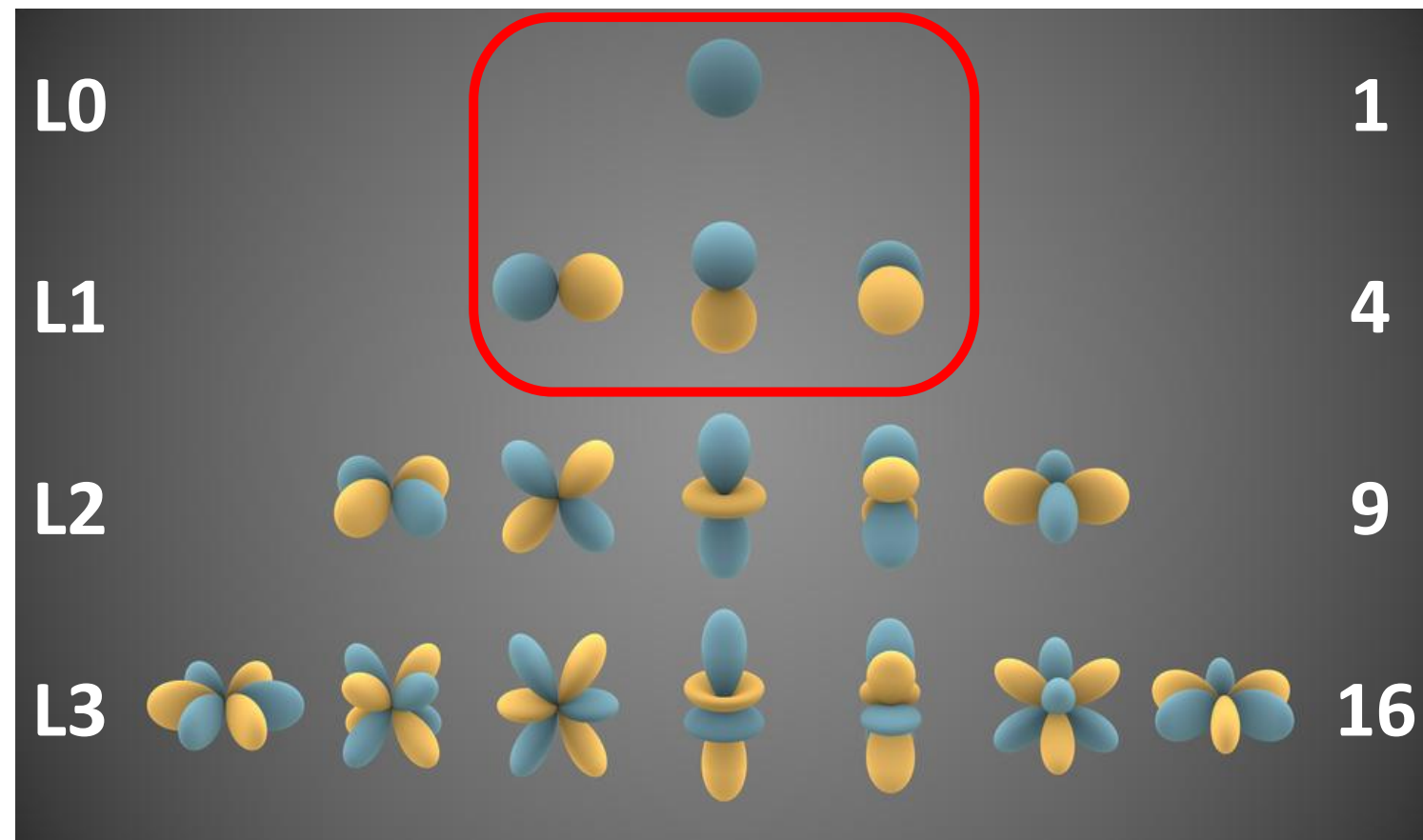
Why DDGI uses SH:

- **Compact:** L1 SH encodes the full directional radiance distribution in **only 4 x 3(rgb) floats** (vs. 192 floats for an 8×8 texel grid)
- **Fast lookup:** evaluating irradiance for any surface normal N reduces to a single dot product — **no texture sampling or numerical integration needed**
- **Linear blending:** SH coefficients can be **blended** with a **simple weighted sum**
- Sufficient for **diffuse**: diffuse radiance is a **low-frequency** directional signal - **L1 SH** captures it accurately enough

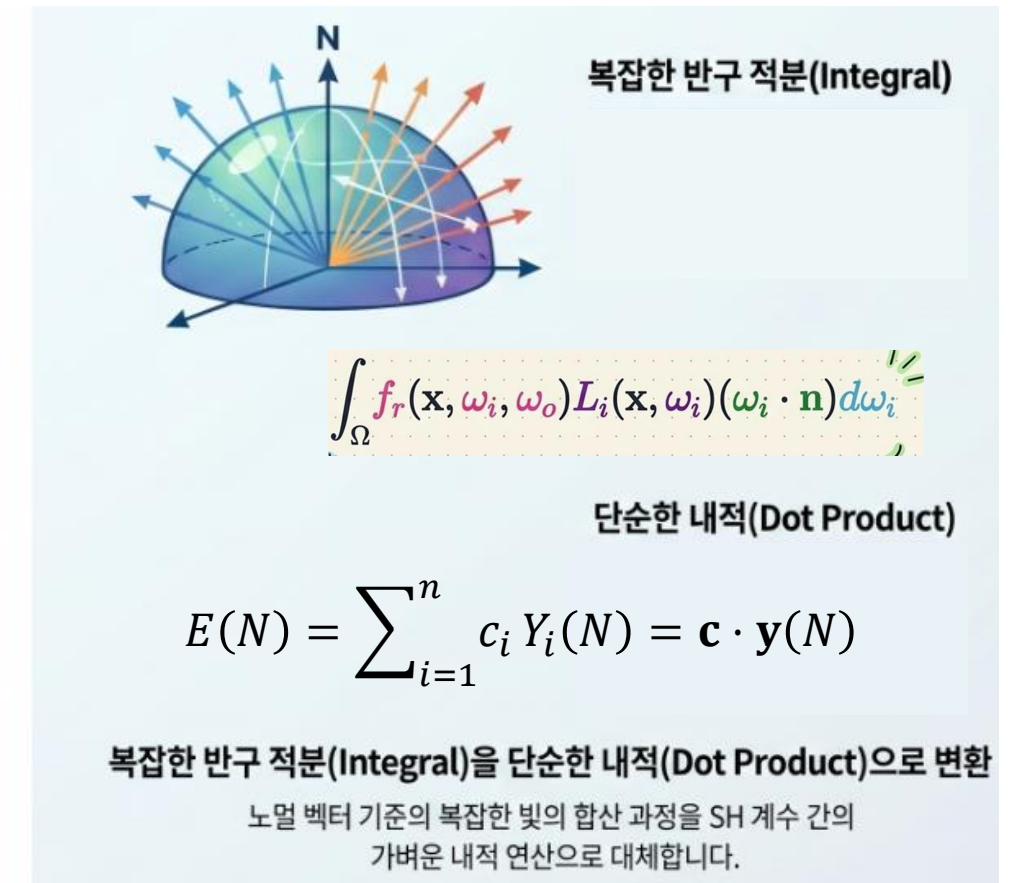
Dynamic Diffuse Global Illumination

Why DDGI uses SH:

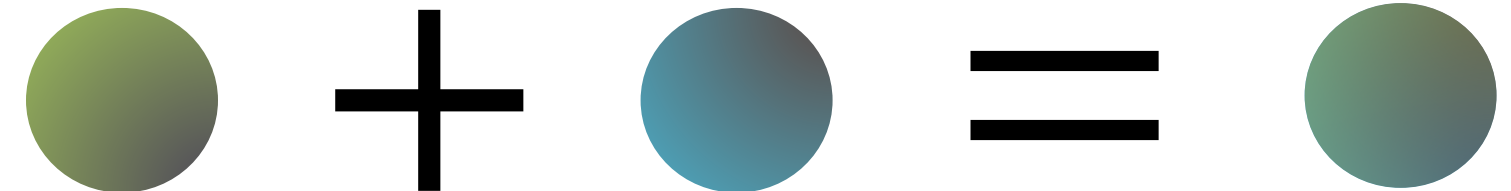
- **Compact:** L1 SH encodes the full directional radiance distribution in **only 4 x 3(rgb) floats** (vs. 192 floats for an 8×8 texel grid)
- **Fast lookup:** evaluating irradiance for any surface normal N reduces to a **single dot product** — no texture sampling or numerical integration needed
- **Linear blending:** SH coefficients can be **blended** with a **simple weighted sum** (interpolation)
- Sufficient for **diffuse**: diffuse radiance is a **low-frequency** directional signal - **L1 SH** captures it accurately enough



방향(Vector)을 입력하면 색상(Color)이 출력되는 구체 함수
SH 구체 표면의 특정 방향 벡터를 입력값으로 넣으면
해당 방향의 색상값을 즉시 얻을 수 있습니다.



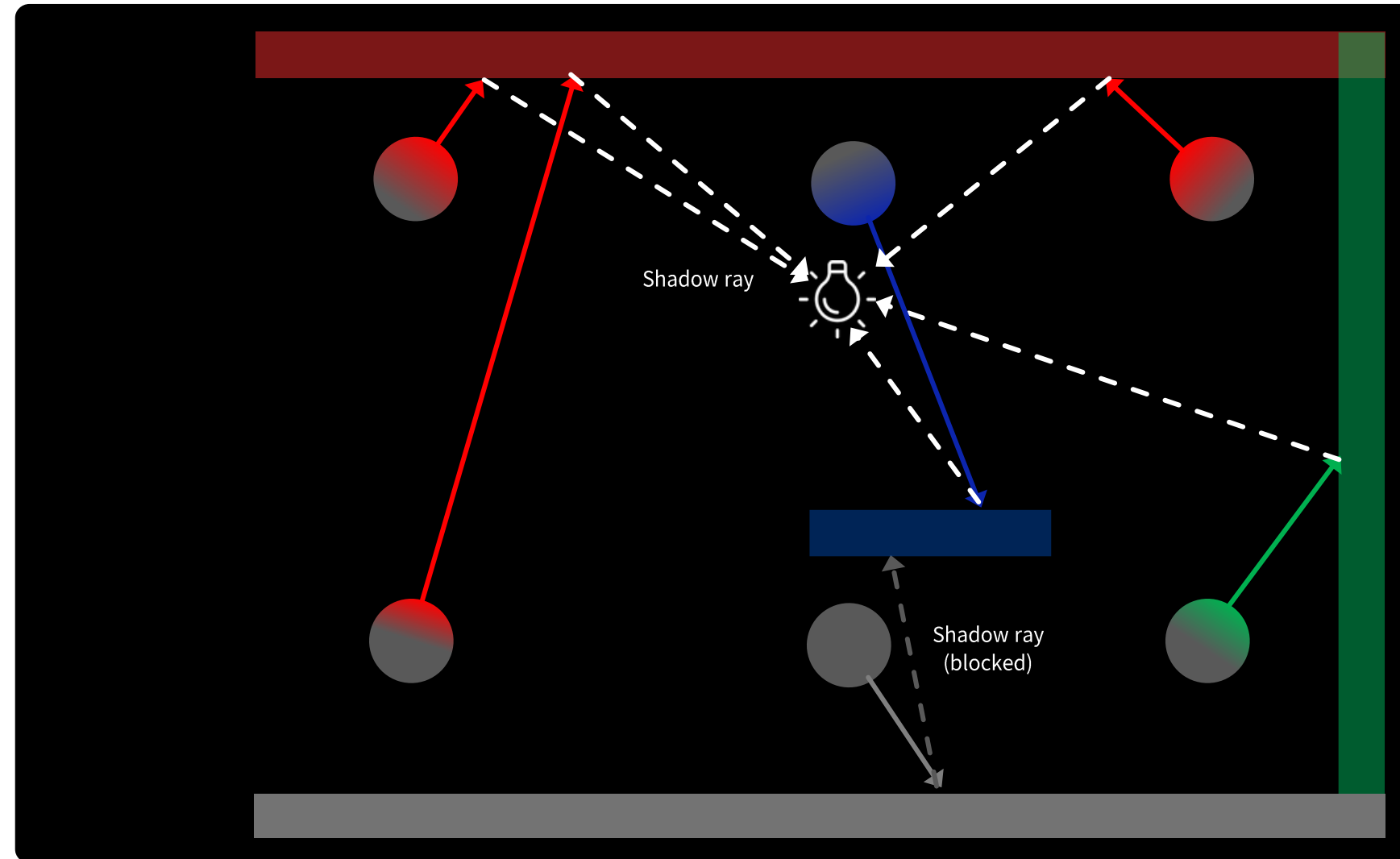
Coefficients (level L) : $\sum_{l=0}^L (2l + 1) = (L + 1)^2$



Weighted sum of coefficients
(interpolation)

Dynamic Diffuse Global Illumination

direct radiance



First frame in DDGI compute shader : no indirect radiance stored in probes yet (cold start)

Each probe fires **N rays** distributed uniformly across the sphere (Spherical Fibonacci sampling + random rotation per frame)

On ray hit : compute **direct radiance** at the **hit surface**

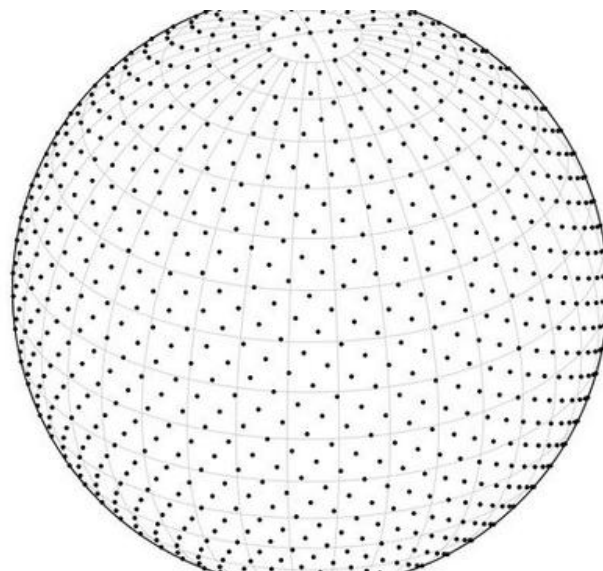
- sample a light source, compute the radiance at the hit surface ($\mathbf{N} \cdot \mathbf{L}$)
- Also store hit distance per direction — used to detect and suppress contribution of occluded probes

$$\int_{\Omega} f_r(\mathbf{x}, \omega_i, \omega_o) L_i(\mathbf{x}, \omega_i) (\omega_i \cdot \mathbf{n}) d\omega_i$$

Cast **shadow ray** toward the light source

- if occluded, discard the contribution

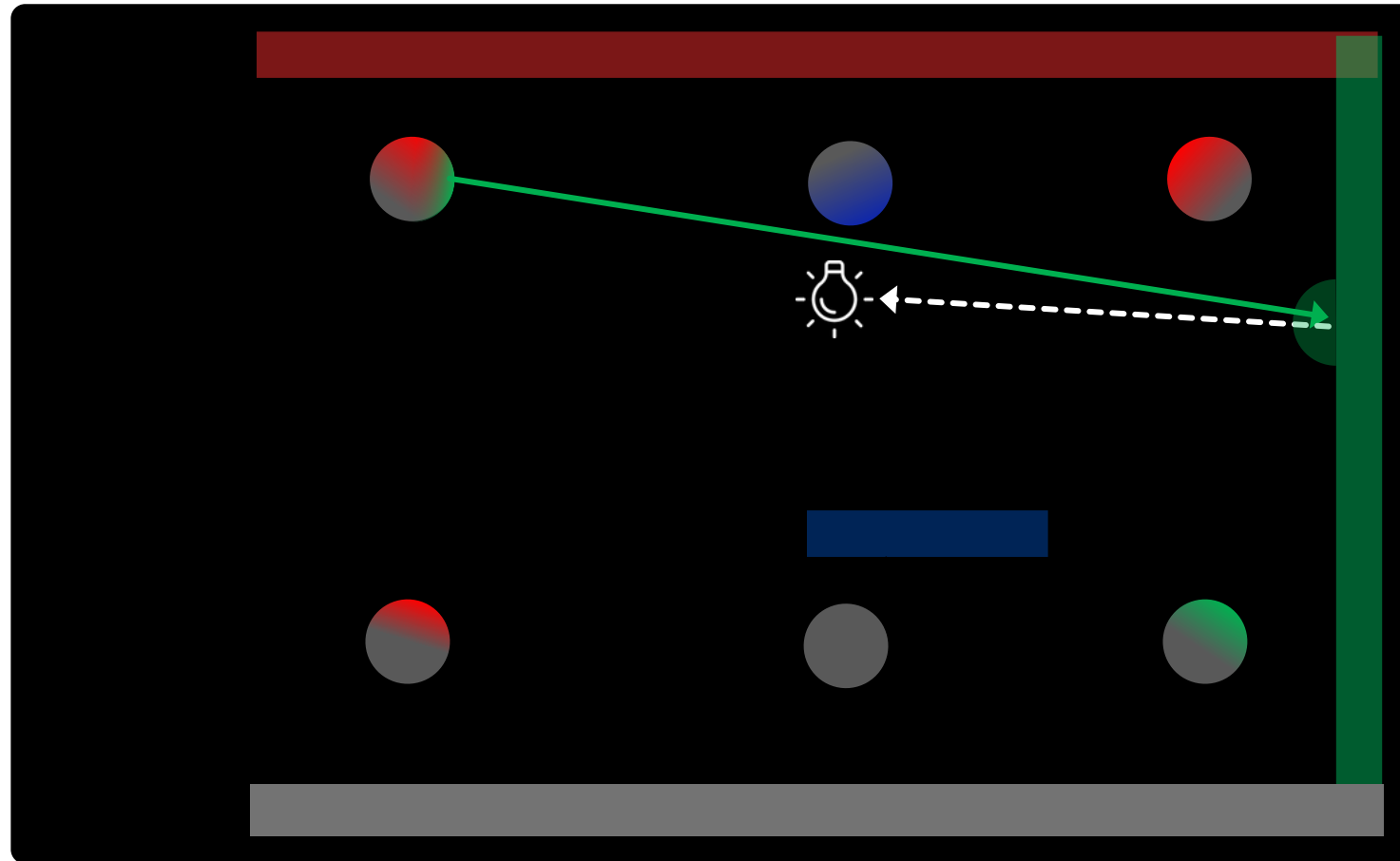
Store accumulated ray results into per-probe **L1 SH coefficients**



Spherical Fibonacci : A deterministic method to distribute N points uniformly across the sphere surface, avoiding clustering.

Dynamic Diffuse Global Illumination

direct radiance



Frames after first frame in DDGI compute shader : indirect radiance is now available in probes

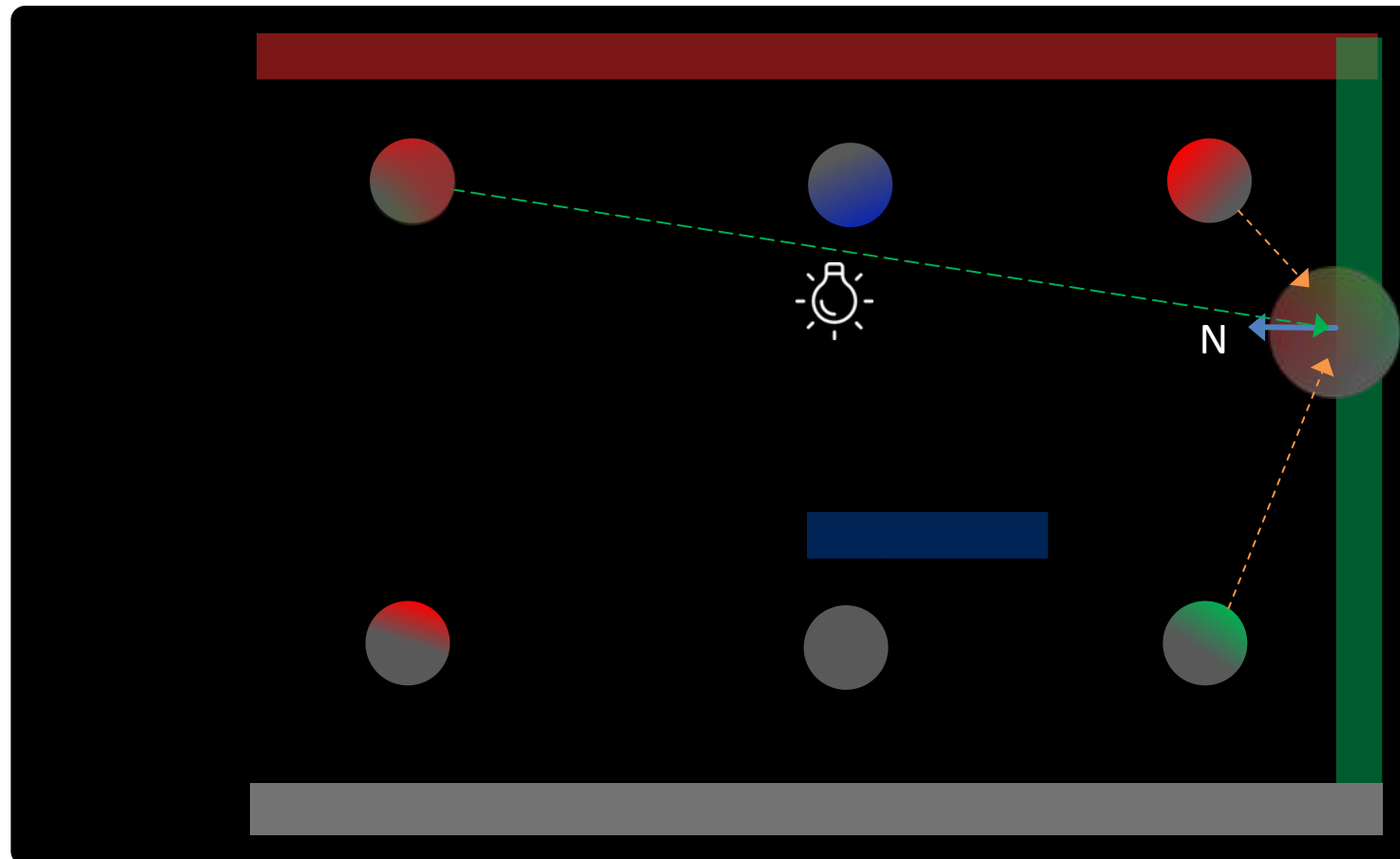
On ray hit: **additionally** sample indirect radiance from nearby probes (in 3D, 8 probes)

- blend probe SH by position, integrate over surface normal $\rightarrow$ indirect diffuse irradiance

Store accumulated ray results (direct + indirect) into per-probe L1 SH coefficients

Indirect light propagates further each frame $\rightarrow$ **infinite bounces converge over time**

indirect radiance

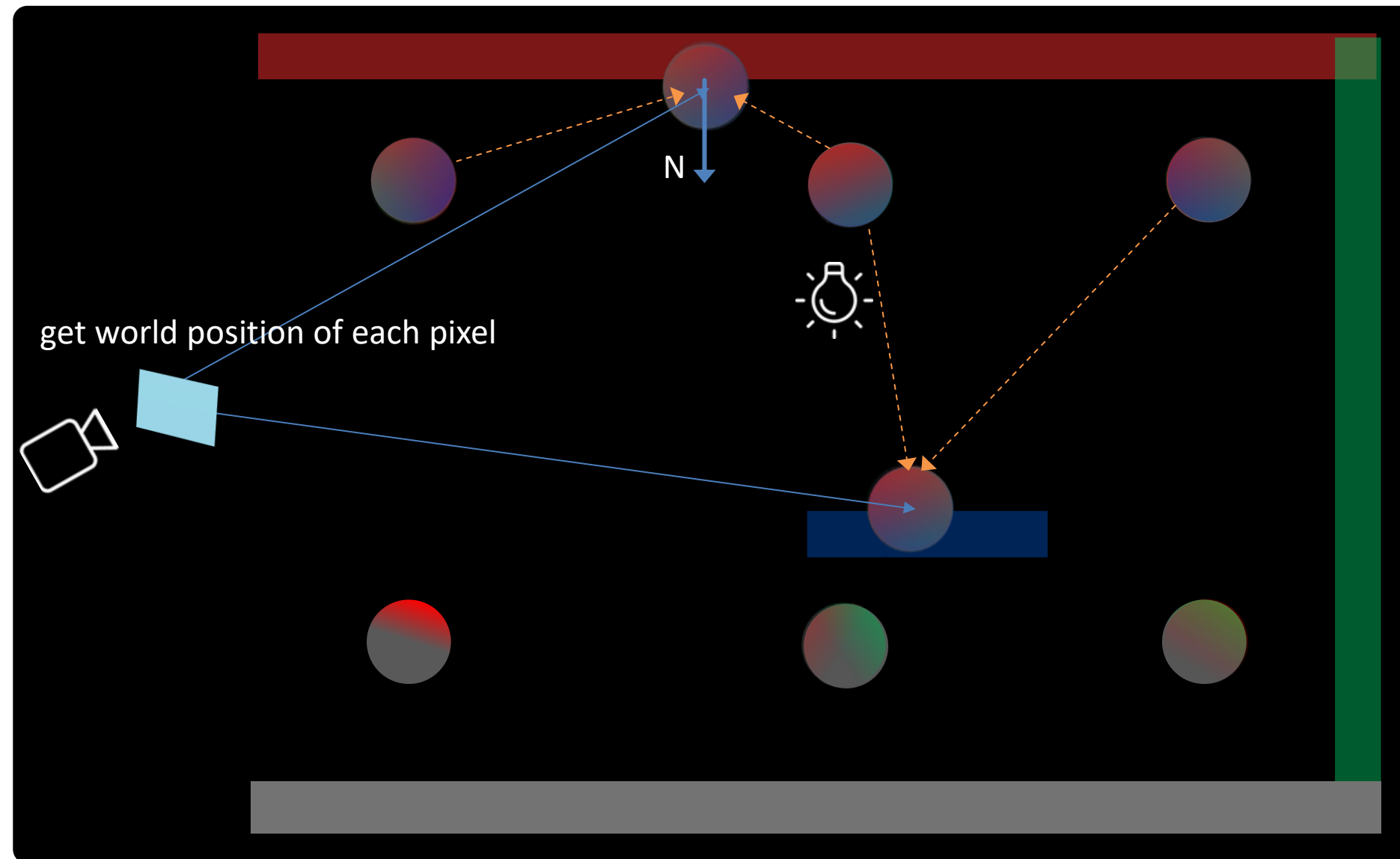


weighted blend near probes's SH



integrate the probe SH over the hemisphere aligned to **surface normal N**
(computable as a simple dot product of SH coefficients)

Dynamic Diffuse Global Illumination

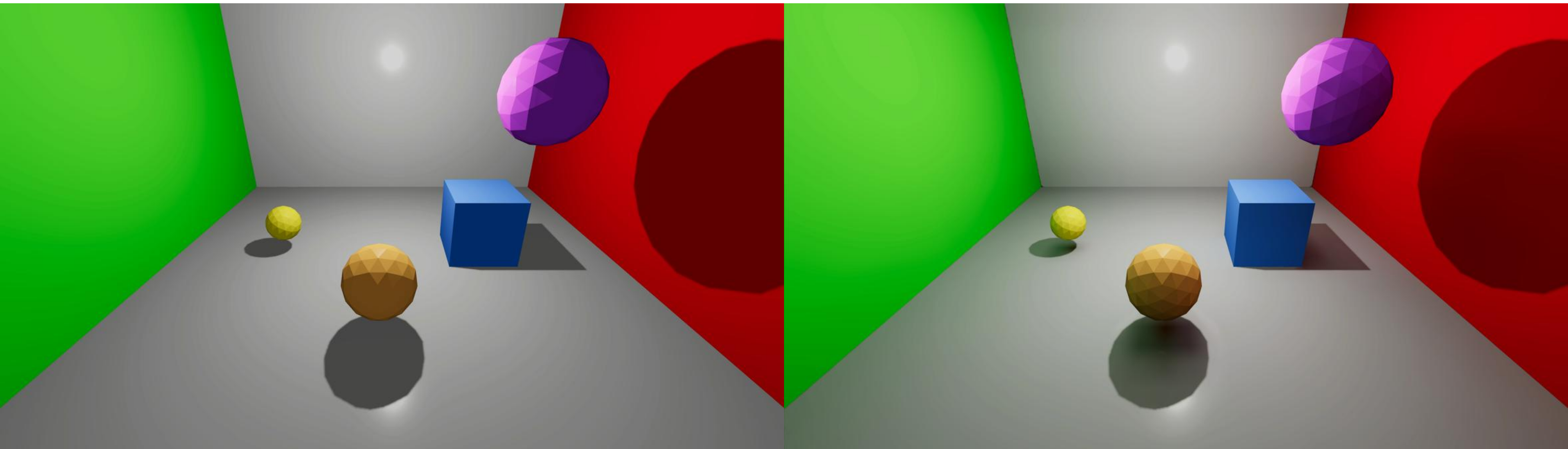


DDGI in the Rasterization Pixel Shader

Now, let's see how the indirect lighting stored in the probes is used during rendering.

1. The pixel shader reconstructs the **world-space position of each pixel**
2. **Indirect lighting** is sampled from **neighboring probes** around the world position, **same as the probe update process**
3. The sampled indirect lighting is combined with the engine's existing lighting model (Phong, PBR, ...) to produce the final shaded result

Dynamic Diffuse Global Illumination



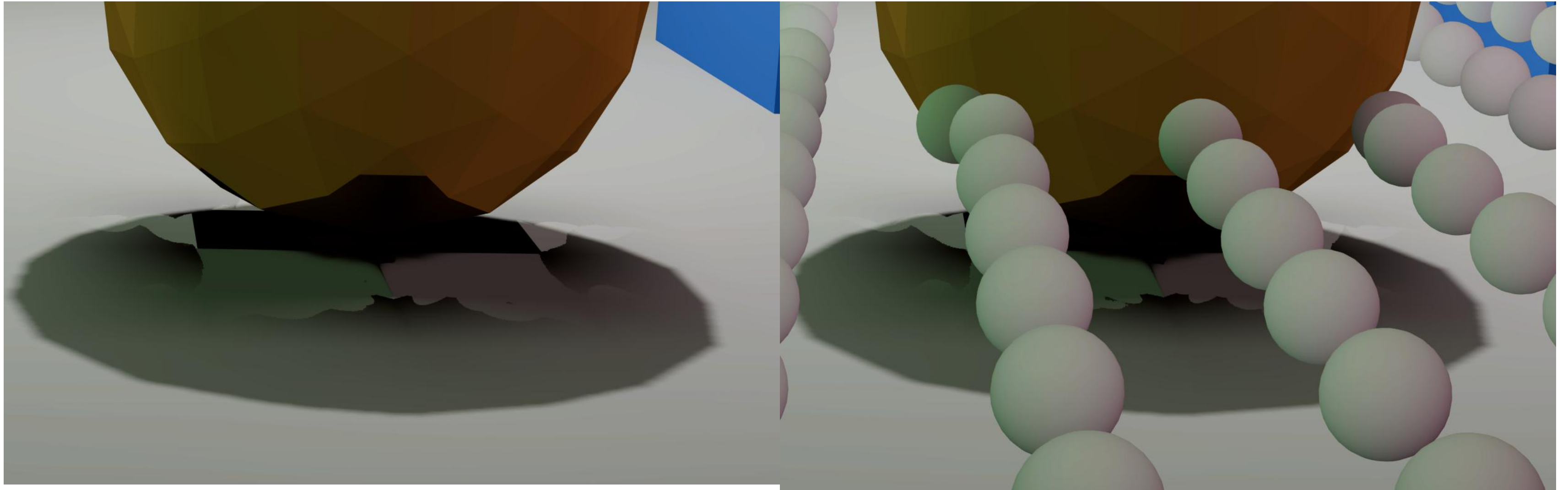
original

DDGI

Strength

- Fully supports **dynamic scenes**; infinite diffuse bounces
- **Compact** per-probe storage via L1 SH (12 floats * grid resolution)

Dynamic Diffuse Global Illumination



Limitation

- Probe **grid resolution** limits fine-detail GI
- L1 SH cannot represent high-frequency lighting — **unsuitable for specular**

